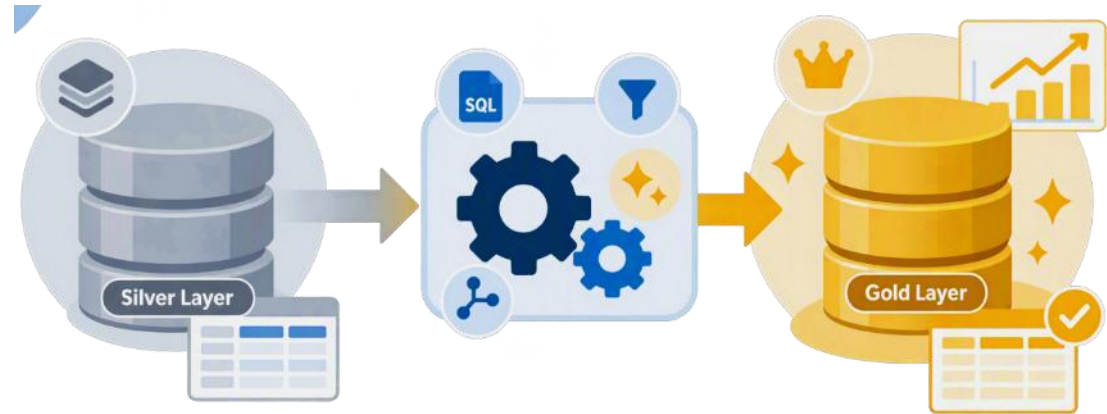


Gold Layer Transformations



Dr. Abdelkarim Erradi
CSE@QU

With assistance from AI tools, particularly
Microsoft Copilot and Google Gemini

Gold Layer: Analytics-Ready with Star Schema

Gold Layer = analytics-ready data: Silver tables reshaped into a **star schema** for dashboards, reports and analytics

Fact Tables (The "What Happened"): records measurable events

It contains **numeric measures** we want to aggregate (sum, average, count) e.g., sales, revenue, quantity sold, cost

Dimension Tables (The "Context"): provide descriptive **context** used to filter, group, and label the facts:

Who?

Customer, Employee

What?

Product, Category

When?

Date, Month, Year

Where?

Store, City, Country

How?

Sales Channel, Payment Method

 **Used to filter and group measures**

Facts tell us WHAT happened. Dimensions explain WHO, WHAT, WHEN, WHERE and HOW it happened.



Why Star Schema

Business-friendly : Analyze data from different perspectives (customer, product, time, and location)

Simple : Facts and dimensions are easy to understand and query. Can answer business questions by simply asking: Who, What, When, Where, and How?

Fast : Simple joins between the fact table and dimensions, with small dimension tables that are efficient to scan and filter.

Consistent : Business metrics (e.g., revenue and profit) are defined once and used everywhere consistently.

Translate a business question into a model

Turn one question into fields

Which products generate the most sales each month?

Each part of the question points to a field:

Sales sum fact_sales.sales_amount

Product dim_product.product_name

Month dim_date.year + month

Sum sales_amount by product and month.

Build the matching star schema

Grain:

one order item per fact row (identified by order_item_id)

fact_sales

Measures: quantity, sales_amount

Order identifier: order_id

Dimension links: date, product_id, customer_id

The dimensions supply labels

Product: product_name, category

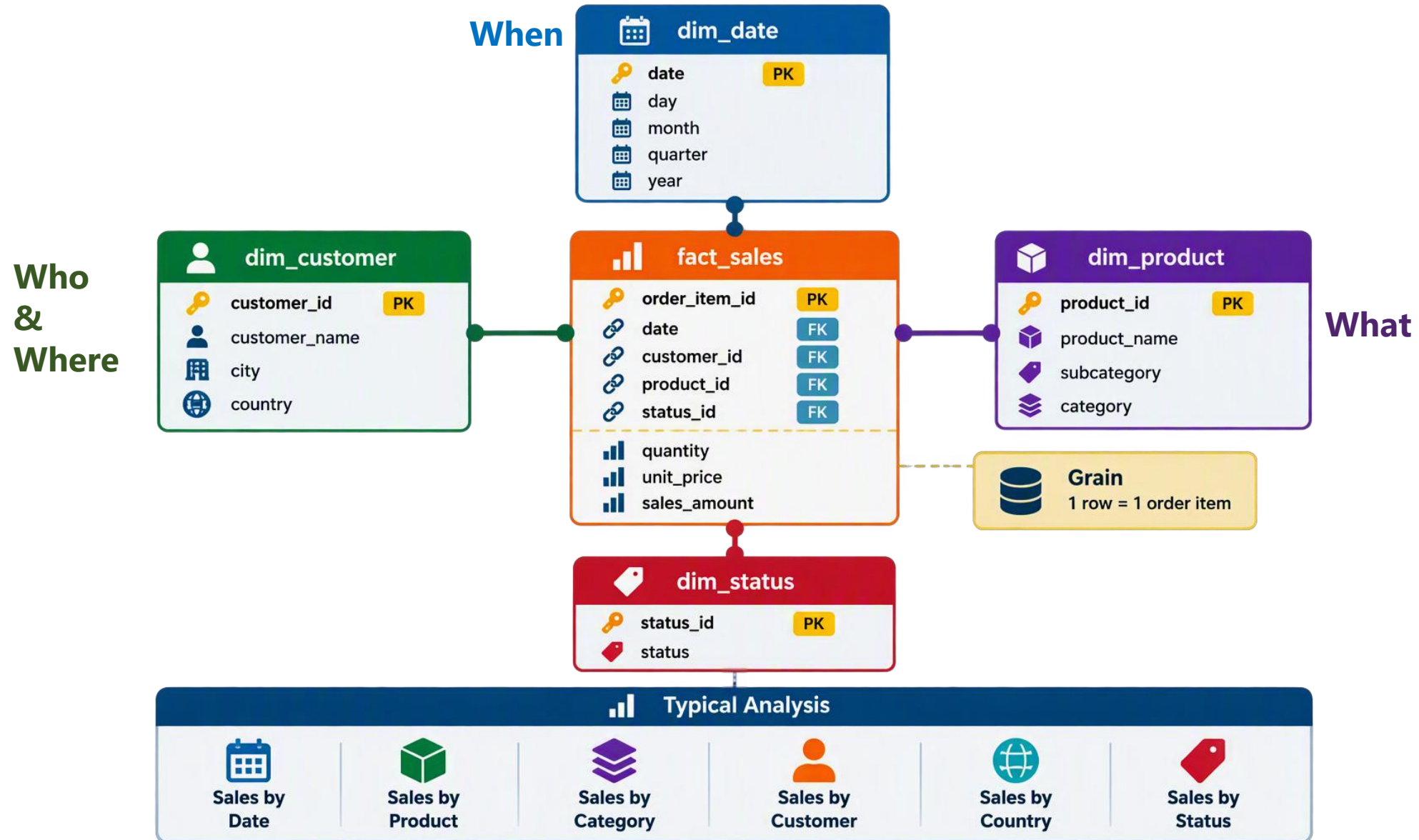
Date: year, month

Answer the question:

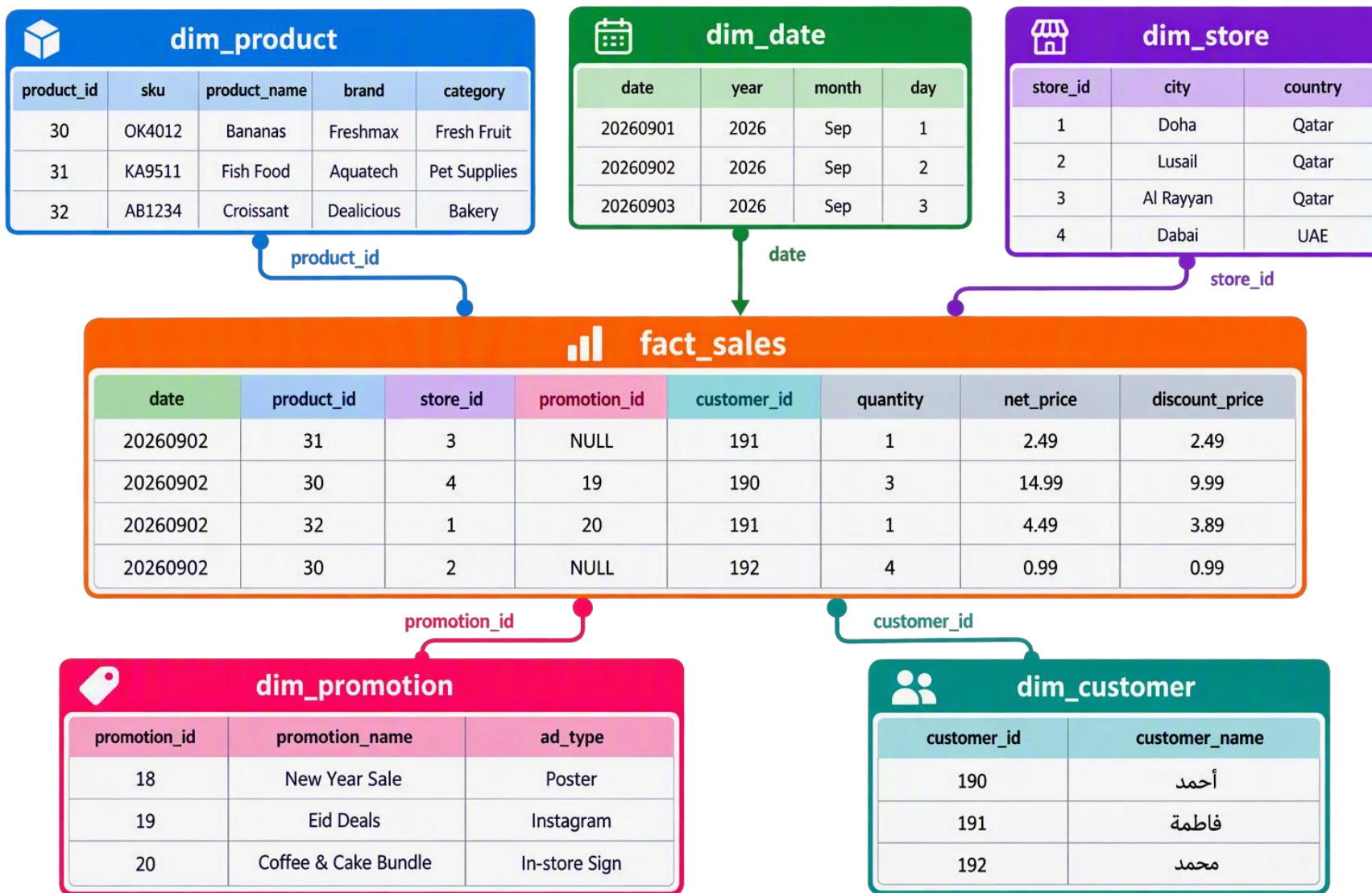
groups sales by month and product, then sorts products by sales within each month

```
SELECT d.month, p.product_name, SUM(f.sales_amount) AS total_sales
FROM fact_sales f JOIN dim_product p ON f.product_id = p.product_id
JOIN dim_date d ON f.date = d.date
GROUP BY d.month, p.product_name
ORDER BY d.month, total_sales DESC;
```

Star Schema Example – analyze sales from several perspectives



Star Schema Another Example



Build a star schema in three steps



1

Define the fact grain

What does row in the fact table represents?

- e.g., One fact row = one order line. Unique identifier **order_item_id**
- Use the lowest (most detailed) grain possible for the primary fact table => **to allow maximum flexibility of analysis**
- Keep every measure at that grain. (e.g., do not mix order lines with daily totals in the same fact table)

2

Create dimensions and their hierarchies

What can we group it by?

- Identify the dimensions that apply to the chosen grain. Give each dimension a **unique key** and include attributes used for **filtering or grouping**.
- Define its **hierarchy** and roll-up attributes
E.g., Product dimension:
Product → Subcategory → Category
Running Shoe → Athletic Footwear → Footwear

3

Create the fact table

What can we measure?

The fact table consists of:

- a **grain identifier** (e.g., order_item_id).
- numeric measures** (e.g., quantity and sales_amount)
- foreign keys** linking to the dimension tables (e.g., customer_id, product_id, date).

One fact_sales row: dimension keys provide context, and measures provide the numbers

customer_id

product_id

date

store_id

quantity

sales_amount

DIMENSION KEYS · who / what / when / where

MEASURES · numbers to add

Key principle: One fact table, one grain. Use a separate fact table for data at a different grain

Choosing the grain and dimensions

Grain tells us what one fact row means. Dimensions describe that same row.

1. Choose the grain:

e.g., one `fact_sales` row = one order line

- Identified by `order_item_id`
- Store measures for **that line**, e.g., quantity and sales_amount.

2. Choose dimensions:

Ask how analysts will group or filter sales: by **customer** (who), **product** (what), **order date** (when), or **store** (where).

- Define hierarchies, such as **Product** → **Subcategory** → **Category**.
- Give each dimension row a unique key so fact rows can link to it.

Which dimensions can each fact use?

Yes means the planned fact keeps that dimension's detail.

Fact / grain	Date	Customer	Product	Status
fact_sales order line	Yes	Yes	Yes	Yes
sales_summary day + product	Yes	No	Yes	No

The daily summary has no Customer or Status detail, so those totals cannot be filtered by either.



Design check: each dimension key identifies one matching row, so joins never multiply fact rows.

Date dimension: calendar context

One row per calendar day gives each sale reusable labels for grouping and filtering by time.

Two ways to choose the dates

Only dates observed in orders

Use distinct order dates when missing days do not matter.

Continuous calendar (recommended)

Generate every calendar day between first to last order.
Allows left-joins to spot days with zero sales.
Delivers accurate average daily sales metrics.

Continuous calendar in SQL

```
WITH bounds AS (  
    -- Find the first and last order dates.  
    SELECT  
        MIN(order_date) AS first_date, MAX(order_date) AS last_date  
    FROM silver.orders  
)  
,  
days AS (  
    -- Generate every calendar day in that range, including days with no orders.  
    SELECT day_ts AS date  
    FROM bounds  
    CROSS JOIN generate_series(  
        first_date, last_date, INTERVAL 1 DAY  
    ) AS generated(day_ts)  
)  
SELECT date, year(date) AS year, month(date) AS month, ...  
FROM days ORDER BY date;
```

Each date carries its context

date, year, quarter, month, month_name,
day_of_week, week_of_year, is_holiday, is_weekend

Example row

2026-09-18, 2026, Q3, September, Friday

How analysts use it

Filter year = 2026 and month = 9, then sum sales_amount.

A left join dim_date with fact_sales also returns days with no sales.

Dimension Design: Leveraging Hierarchies

Hierarchies enable analysts to **summarize** numbers at different levels while the fact remains at the lowest grain.



The **dim_product** hierarchy lets analysts sum **sales_amount** by **product**, then roll it up to **subcategory** or **category**, while **fact_sales** remains at the order-line grain.



The **dim_location** hierarchy lets analysts sum **sales_amount** by **city**, then roll it up to **country** or **GCC**, while **fact_sales** remains at the order-line grain.

Common Analytical Operations

A Star Schema enables **self-service analytics** in tools such as Power BI, Tableau, and Excel, allowing analysts to seamlessly perform key analytical operations:

↑ **Roll Up:** Aggregate metrics to a higher hierarchy level to get the big picture.

✖ *E.g.,:* Moving from sales_amount by City (Doha, Al Wakrah) to sales_amount by Country (Qatar)

+ **Drill Down:** Navigate into details from an aggregated view to find root causes..

✖ *E.g.,:* Clicking on "Footwear" to reveal sales for "Athletic Footwear" and "Outdoor Footwear".

✂ **Slice:** Filtering the dataset by a *single* dimension to analyze a subset.

▼ *E.g.,:* WHERE Country = 'Qatar'

🎲 **Dice:** Filtering the dataset by *multiple* dimensions to isolate a specific segment.

📦 *E.g.,* WHERE Country = 'Qatar' AND Category = 'Footwear'

Star schema enables analytics operations: rollup, drilldown, slice, and dice



Product hierarchy

Roll up from product to subcategory to category.



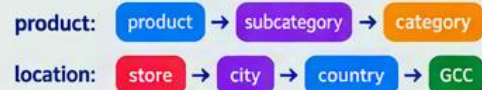
Location hierarchy

Roll up from store to city to country to GCC.

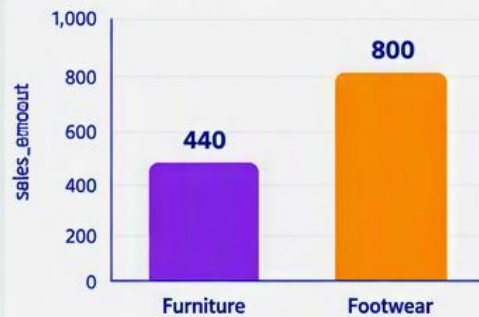


Roll up

Aggregate by product category.



Sales by Product Category



Roll up to get a bigger picture (e.g., by category or GCC).

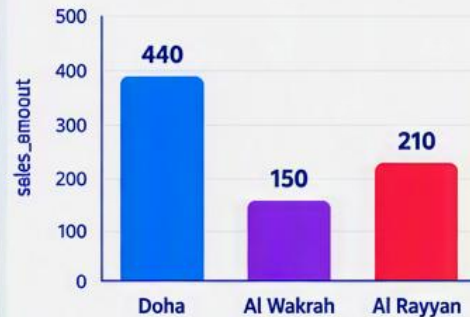


Drill down

Navigate to details.



sales_amount by city (Qatar)



Drill down to see more detail (e.g., from country to store).

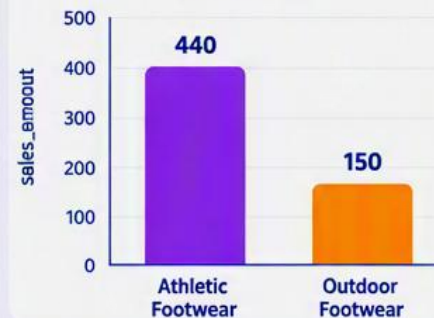


Slice

Filter by one dimension.



sales_amount by subcategory (Qatar only)



Slice to focus on a subset (e.g., one country).

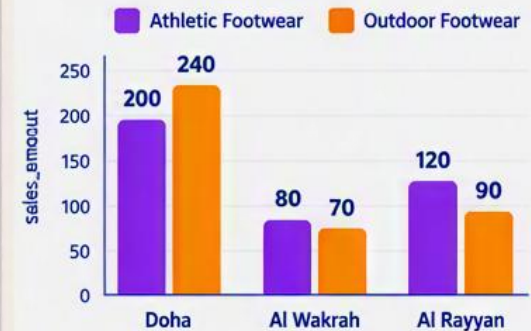


Dice

Filter by multiple dimensions.



sales_amount by city and subcategory (Qatar, Footwear)



Dice to analyze a specific segment (e.g., Qatar + Footwear).



Key takeaway

The level of aggregation changes, but the fact table remains at the same grain (e.g., one order line).



Supported by hierarchies Works for product and location hierarchies (and others).

Practical Implementation

Load dimensions and facts & handle incremental updates
using SQL and Pandas

Build customer dimension using SQL

Customer gives the “who” context for every sale.

CUSTOMER - Upsert changed records

```
MERGE INTO gold.dim_customer t
USING silver.customers s
ON t.customer_id = s.customer_id
WHEN MATCHED THEN UPDATE SET
    city = s.city,
    customer_name = s.customer_name
WHEN NOT MATCHED THEN INSERT
    (customer_id, city, customer_name)
VALUES (s.customer_id, s.city, s.customer_name);
```

Load and validate dimensions before inserting facts that reference them.

Build customer dimension using Pandas

Load the current Gold tables first, then apply a customer upsert

CUSTOMER: latest values win

```
assert silver_customers.customer_id.is_unique

customer_delta = silver_customers[
    ['customer_id', 'customer_name', 'city']]
dim_customer = pd.concat(
    [dim_customer, customer_delta],
    ignore_index=True
).drop_duplicates('customer_id', keep='last')
```

Upsert fact_sales at the declared grain

- **Incremental behavior:** MERGE INTO update existing lines and insert new lines based on the grain identifier (order_item_id).
- **Idempotent data pipeline:** running the same load multiple times produces the same exact result without appending duplicates.

```
MERGE INTO gold.fact_sales AS t
USING (SELECT i.order_item_id,
            o.order_date AS date, o.customer_id,
            i.quantity, i.unit_price,
            i.quantity * i.unit_price AS sales_amount
        FROM silver.order_items AS i
        JOIN silver.orders AS o ON i.order_id = o.order_id
        WHERE o.order_date <= DATE {run_date}
    ) AS s ON t.order_item_id = s.order_item_id
WHEN MATCHED THEN UPDATE SET
    date = s.date, customer_id = s.customer_id,
    quantity = s.quantity, unit_price = s.unit_price, sales_amount = s.sales_amount
WHEN NOT MATCHED THEN INSERT
    (order_item_id, date, customer_id, quantity, unit_price, sales_amount)
VALUES (s.order_item_id, s.date, s.customer_id, s.quantity, s.unit_price, s.sales_amount);
```

Load the sales fact in Pandas

Use the same line-item grain, date and customer keys, and calculated sales amount.

```
lines = silver_order_items.merge(  
    silver_orders, on='order_id',  
    how='inner', validate='many_to_one')  
assert lines.order_item_id.is_unique  
lines['sales_amount'] = (  
    lines.quantity * lines.unit_price)  
  
fact_sales = pd.concat([fact_sales, lines])  
fact_sales = fact_sales.drop_duplicates(  
    'order_item_id', keep='last')
```

Aggregate to produce sales summary

For high-frequency querying we can build derivative summary tables (e.g., sales_summary).

DAILY PRODUCT SUMMARY

```
CREATE OR REPLACE TABLE gold.sales_summary AS
SELECT date, product_id,
       SUM(quantity) AS units_sold,
       SUM(sales_amount) AS sales_amount,
       COUNT(DISTINCT order_id) AS order_count
FROM gold.fact_sales
GROUP BY date, product_id;
```

Declared grain

One row per date + product_id.

Cannot slice this table by customer or status.

Use COUNT(DISTINCT order_id) because an order with multiple line items **appears in multiple fact rows** but should count as one order.

Use summaries only for high-frequency analytical queries.

Fact Table Design: Granularity & Summaries



Base Fact: fact_sales

Grain: One Order Item (order line)

GRAIN IDENTIFIER

order_item_id

FOREIGN KEYS (CONTEXT)

date customer_id product_id status

MEASURES (NUMBERS)

quantity unit_price sales_amount

WHEN TO USE

Customer and/or product level analysis



Highly flexible. Can answer any question involving Customer, Product, Date, or Status.

Aggregated: sales_summary

Grain: Date + Product (Daily summary)

GRAIN IDENTIFIER date + product_id

FOREIGN KEYS (CONTEXT)

date product_id *(Customer & Status dropped)*

AGGREGATED MEASURES

SUM(quantity) AS units_sold

SUM(sales_amount) AS sales_amount

COUNT(DISTINCT order_id) AS order_count

WHEN TO USE

Daily product sales dashboard



Use COUNT(DISTINCT) because one order can have multiple line items. Used for high-frequency dashboards.

Summary & Key Takeaways

- **Gold = Business-Ready:** Organize data for analysts and self-service analytics.
- **Grain is King:** Define exactly what **one fact row represents** before designing the fact and dimensions.
 - E.g.,: One fact_sales row = **one order line**, identified by order_item_id. Keep all measures at this grain.
- **Dimensions Add Context:** Build dimensions and their hierarchies to describe the facts.
 - dim_product identifies the product; dim_date provides attributes such as month and year.
 - Each fact foreign key should match **exactly one dimension row**.
- **Gold Design Checklist:**

• Define one clear grain per fact. • Derive aggregates from the base fact.	• Ensure dimension keys are unique. • Make loads idempotent and repeatable.
--	---
- **Idempotency:** Running the pipeline multiple times should produce the **same result without creating duplicates**. Use MERGE in SQL or drop_duplicates() in Pandas where appropriate.

References

Dimensional modeling

Kimball Group

- Define exactly what one fact row represents ([Grain](#))
- Choose descriptive attributes and keys ([Dimension Table Structure](#))
- Design reusable date dimension ([Calendar Date Dimensions](#))

SQL and Pandas implementation

DuckDB documentation

- Generate a continuous range of dates in SQL using [generate_series](#) function
- Update or insert rows during incremental loads using [MERGE INTO Statement](#)

Pandas documentation

- Join tables using [DataFrame.merge](#)
- Keep the latest row for each business key using [DataFrame.drop_duplicates](#)